Docker Notes By Adnan Qureshi

#Day-01 (Basics & Foundations)

What is Docker?

- **Docker**: An open-source platform to build, ship, and run applications in isolated environments called **containers**.
- Containers package your app + dependencies → runs the same everywhere (local, server, cloud).
- Think of it as a **lightweight alternative to virtual machines (VMs)**.

Why Docker?

- Consistency across environments ("works on my machine" → solved).
- Faster than VMs (shares OS kernel instead of full OS).
- Easy scaling & deployment.
- Huge ecosystem (Docker Hub, Compose, Swarm, Kubernetes).

Key Concepts

- **Image**: Blueprint of an app (like a snapshot). Example: nginx:latest
- **Container**: Running instance of an image.
- **Docker Engine**: The runtime that runs containers.
- **Docker Hub**: Public registry of images.

Installing Docker

- Install from docker.com.
- Verify installation: with `docker –version` and `docker info` commands.

First Hands-on

Run hello-world: `docker run hello-world`

Run a container: `docker run nginx`

List running containers: `docker ps`

List all containers (including stopped): `docker ps –a`

Stop and remove container: `docker stop <container_id>` `docker rm <container_id>`

Common Container Commands

Command	Purpose
docker run <image>	Run container from image
docker run -it ubuntu bash	Interactive terminal in container
docker exec -it <id> bash	Run command inside running container
docker stop <id>	Stop container
docker start <id>	Start stopped container
docker rm <id>	Remove container
docker logs <id>	Show container logs

Practice for Week 1:

1. Run Ubuntu container and explore shell.
2. Run Nginx container, access it in browser (localhost:80).
3. Stop and restart containers.

#Day-02 (Working with images)

Docker Images & Layers

- Image = read-only template.
- Built from layers → each RUN COPY, etc. in Dockerfile creates a new layer.
- Containers are writable layers on top of images.

Important Image Commands

Command	Purpose
docker images	List images
docker pull <image>	Download image from Docker Hub
docker rmi <image>	Remove image
docker tag <src> <repo>:<tag>	Tag image with new name
docker push <repo>:<tag>	Push image to registry

Writing a Dockerfile

A Dockerfile is a set of instructions to build an image.

Example: Python App

```
# Base image
FROM python:3.9-slim
```

```
# Set working directory
WORKDIR /app
```

```
# Copy files
COPY requirements.txt .
RUN pip install -r requirements.txt
```

```
COPY . .
```

```
# Expose port
EXPOSE 5000
```

```
# Command to run
CMD ["python", app.py”]
```

Building & Running Images

Build image: `docker build -t myapp:1.0 .`

Run container: `docker run -p 5000:5000 myapp:1.0` (maps container port → host port)

Layers & Caching

- Docker caches layers → faster builds.
- Place frequently changing code (`COPY . .`) at the bottom of Dockerfile.

Best Practices

- Use lightweight base images (alpine).
- Pin versions (python:3.9-slim, not just latest).
- Keep images small → remove unnecessary files.

Useful Image Commands

Command	Purpose
docker history <image>	Show image layers
docker inspect <image>	Detailed metadata
docker save <image> -o file.tar	Export image
docker load -i file.tar	Import image

Practice for Day 2:

1. Pull python:3.9, ubuntu, and nginx images.
2. Write Dockerfile for a Python Flask app.
3. Build your image and run it with -p port mapping.
4. Push image to Docker Hub (need Docker Hub account).

#Day-03 (managing containers)

Container Lifecycle

A container goes through these states:

- **Created** → when docker create is used (not running yet).
- **Running** → actively executing (docker start or docker run).
- **Paused** → temporarily suspended.
- **Stopped/Exited** → process ended.
- **Removed** → deleted from system.

Commands:

```
docker create nginx              # Creates container but doesn't start it
docker start <container_id>      # Start created/stopped container
docker restart <container_id>    # Restart container
docker stop <container_id>       # Gracefully stop container
docker kill <container_id>       # Force stop
docker rm <container_id>         # Remove container
```

Running Containers: Detached vs Interactive

Detached mode (-d) → runs in background : docker run -d nginx

Interactive mode (-it) → runs with attached terminal docker run -it ubuntu bash

Attach to a running container docker attach <container_id>

◆# Executing Commands in Containers

- Run commands in a running container:

  ```
  docker exec -it <container_id> bash
  docker exec -it <container_id> ls /app
  ```

- Difference between attach vs exec:
  - attach: connects to main process → can close container accidentally if you exit.
  - exec: runs new command without disturbing main process.

Data Persistence with Volumes

Containers are **ephemeral** → once removed, data inside is lost.
To persist data, Docker provides **Volumes**.

Types of Volumes

1. **Anonymous Volume** – created automatically when not named.

   ```
   docker run -v /data mysql
   ```

2. **Named Volume** – user-defined & reusable.

   ```
   docker volume create mydata
   docker run -v mydata:/var/lib/mysql mysql
   ```

3. **Bind Mounts** – link host directory to container.

   ```
   docker run -v /host/path:/container/path nginx
   ```

Volume Commands

```
docker volume ls          # list volumes
docker volume inspect mydata
docker volume rm mydata
```

Docker Networks

Containers can communicate using **Docker networks**.

Default Networks

- **bridge**: default for standalone containers.
- **host**: container shares host's network.

- **none**: no networking.

Custom Bridge Network

```
docker network create mynet
docker run -d --name db --network mynet mysql
docker run -d --name app --network mynet myapp
```

Now app can reach db using the name db (DNS-based).

Network Commands

```
docker network ls
docker network inspect mynet
docker network rm mynet
```

Port Mapping

By default, containers are isolated from the host.
We expose container services using **port mapping** (-p).

```
docker run -d -p 8080:80 nginx
```

- Host 8080 → Container 80.
- Access via http://localhost:8080.

Environment Variables

Pass env vars to containers using -e:

```
docker run -e MYSQL_ROOT_PASSWORD=secret mysql
```

Or with .env file:

```
docker run --env-file .env myapp
```

Logs & Debugging

View logs: `docker logs <container_id>` `docker logs -f <container_id> # follow logs`

Inspect container: `docker inspect <container_id>`

Resource usage: `docker stats`

Resource Limits

Limit CPU & Memory to avoid resource hogging.

```
docker run -m 512m --cpus="1.0" nginx
```

- -m = memory limit (512 MB)
- --cpus = number of CPU cores

10. Clean Up Commands

Stop all containers: `docker stop $(docker ps -q)`

Remove all containers: `docker rm $(docker ps -aq)`

Remove unused volumes & networks: `docker system prune`

Practice for Week 3

1. Run Ubuntu interactively, install a package (`apt-get install curl`).
2. Create MySQL container with named volume to persist data.
3. Create custom network, run `db` and `app` containers inside it, make them communicate.
4. Run Nginx with port mapping (`8080:80`) and access in browser.
5. Limit container resources (`512MB RAM, 1 CPU`) and check with `docker stats`.

Summary (Week 3)

- Container lifecycle: create, start, stop, restart, remove.
- Interactive (`-it`) vs Detached (`-d`) mode.
- Persistent storage with **Volumes**.
- Networking: default & custom bridge networks.
- Port mapping (`-p`), environment variables (`-e`).
- Logs, inspection, debugging, and resource limits.

#Day-04 (Docker Compose (Multi-container Apps))

What is Docker Compose?

- **Docker Compose** is a tool to define and run **multi-container applications** using a single file `docker-compose.yml`).
- Instead of running multiple `docker run` commands manually, you describe everything in one YAML file.
- Example use case: **Web App + Database + Cache** (all started with one command).

Installing Docker Compose

- Included in **Docker Desktop**.
- Verify: docker compose version # (new syntax)
- Note: Old syntax was docker-compose now it's docker compose.

Structure of docker-compose.yml

Basic structure:

```
version: "3.9"                # Compose file format
services:                     # Define services (containers)
  app:                        # Service name
    image: nginx               # Image to use
    ports:
      - "8080:80"             # Host:Container port
    volumes:
      - ./app:/usr/share/nginx/html
```

Important Commands

Command	Purpose
docker compose up	Start all services
docker compose up -d	Start in detached mode
docker compose down	Stop and remove containers, networks
docker compose ps	Show running services
docker compose logs	Show logs of all services
docker compose exec <service> <cmd>	Run command inside service container
docker compose build	Build images defined in file

Multi-Container Example (Web + Database)

docker-compose.yml

```
version: "3.9"
services:
  db:
    image: mysql:8.0
    container_name: mydb
```

```
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: myappdb
    volumes:
      - dbdata:/var/lib/mysql
  app:
    build: .
    container_name: myapp
    ports:
      - "5000:5000"
    depends_on:
      - db
    environment:
      - DATABASE_HOST=db
      - DATABASE_USER=root
      - DATABASE_PASSWORD=rootpass
      - DATABASE_NAME=myappdb
volumes:
  dbdata:
```

Here:

- db = MySQL container (with persistent volume).
- app = our custom app (built from Dockerfile).
- depends_on = ensures db starts before app.
- A **network is auto-created** → app can connect to db using hostname db.

Volumes in Compose

You can define named volumes at bottom:

```
volumes:
  dbdata:
```

This ensures persistent data across container restarts.

Networks in Compose

By default, Compose creates a network. You can also define custom ones:

```
networks:
  mynet:
services:
  app:
    networks:
      - mynet
  db:
```

```
  networks:
    - mynet
```

Environment Variables in Compose

Use .env file in the same folder:

```
MYSQL_ROOT_PASSWORD=secret
MYSQL_DATABASE=mydb
```

In docker-compose.yml:

```
environment:
  MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
  MYSQL_DATABASE: ${MYSQL_DATABASE}
```

Building Images in Compose

If your service has a Dockerfile:

```
services:
  app:
    build: .
    ports:
      - "5000:5000”
```

Compose will **build image** before running.

Scaling Services

Run multiple instances of a service:

```
docker compose up --scale app=3
```

Useful for load balancing.

Practice for Week 4

1. Write a Compose file for **Nginx + MySQL**.
2. Containerize a small Python/Node.js app and connect it to MySQL using Compose.
3. Add persistent volume for MySQL.
4. Use .env file to manage secrets (DB password).
5. Try scaling app service to 2-3 containers.

Summary (Week 4)

- **Compose** = YAML file to define multi-container apps.
- docker compose up starts all services together.
- Supports **services, volumes, networks, environment variables**.
- Simplifies running **apps + DB + cache** in one command.
- Useful for local dev, testing, and even small deployments.

#Day-05 (Intermediate concepts)

Environment Variables & .env Files

Passing Variables: docker run -e MYSQL_USER=root -e MYSQL_PASSWORD=secret mysql

Using .env File

.env

```
MYSQL_USER=root
MYSQL_PASSWORD=secret
```

In Compose:

```
services:
  db:
    image: mysql
    environment:
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
```

Best practice: use .env files to avoid hardcoding secrets in docker-compose.yml.

Health Checks

Docker can **monitor container health** and restart unhealthy containers.

Example in Dockerfile:

```
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \
  CMD curl -f http://localhost:5000/health || exit 1
```

Example in Compose:

```
services:
  app:
    build: .
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
```

```
    retries: 3
```

Check status:

```
docker ps
```

Shows (healthy) or (unhealthy).

Logs & Debugging

Logs

```
docker logs <container_id>      # view logs
docker logs -f <container_id>    # follow logs
```

Debugging

```
docker exec -it <container_id> sh
docker inspect <container_id>
docker stats    # live resource usage
```

Resource Limits

Prevent containers from consuming all system resources.

```
docker run -m 512m --cpus="1.5" myapp
```

- `-m 512m` → max 512 MB RAM
- `--cpus="1.5"` → 1.5 CPU cores

In Compose:

```
services:
  app:
    image: myapp
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
```

Security Best Practices

1. **Run as non-root** inside containers.

   ```
   RUN adduser --disabled-password appuser
   USER appuser
   ```

2. Don’t expose unnecessary ports.
3. Keep images minimal (alpine, slim).
4. Regularly scan images: docker scan myimage
5. Don’t store secrets in Dockerfile → use .env or secret managers.

🔷 6. Multi-Stage Builds (Image Optimization)

Helps create **small & efficient images**.

Example: Build a Go app

```
# Stage 1: Build
FROM golang:1.19 AS builder
WORKDIR /app
COPY . .
RUN go build -o main .
```

```
# Stage 2: Runtime
FROM alpine:latest
WORKDIR /app
COPY --from=builder /app/main .
CMD ["./main"]
```

Result: final image only has binary, not full Go compiler.

Docker Ignore (.dockerignore)

Similar to .gitignore, prevents unnecessary files from going into image.

Example .dockerignore

```
node_modules
.git
*.log
```

Keeps images smaller and builds faster.

Container Restart Policies

Ensure containers auto-restart on failure.

```
docker run --restart always nginx
```

Options:

- no (default)
- on-failure (restart only on errors)

- always (always restart)
- unless-stopped

In Compose:

```
services:
  app:
    image: myapp
    restart: always
```

Data Backup & Restore

Export container filesystem

```
docker export <container_id> > backup.tar
docker import backup.tar
```

Export/Import image

```
docker save myimage > myimage.tar
docker load < myimage.tar
```

Backup named volumes

```
docker run --rm -v mydata:/data -v $(pwd):/backup busybox tar cvf /backup/data.tar /data
```

Useful Monitoring Tool

- docker stats → live CPU/mem usage.
- docker top <id> → processes inside container.
- docker events → real-time Docker events.

For production: tools like **Prometheus, Grafana, cAdvisor**.

Practice for Day 5

1. Create .env file for database credentials in Compose.
2. Add a **healthcheck** to your app service.
3. Limit app service to **1 CPU, 512MB RAM** and check docker stats.
4. Write a **multi-stage Dockerfile** (build + runtime).
5. Create .dockerignore and test smaller image size.
6. Run a container with --restart=always and test by stopping Docker → container should restart.

Summary (Day-5)

- Use **environment variables** & .env for config.
- Add **health checks** for reliable services.

- Debug using logs, exec, inspect, stats.
- Limit **CPU & memory** for containers.
- Follow **security best practices** (non-root, minimal images).
- Use **multi-stage builds** to optimize images.
- Automate restarts with **restart policies**.
- Use .dockerignore for efficient builds.

#Day-06 (Advanced concepts & Ecosystem)

Why Orchestration?

So far, you've been running containers on a single machine.
But in **production**, we often need:

- Multiple containers across multiple servers
- Scaling up/down dynamically
- Self-healing (restart if crashed)
- Load balancing
- Service discovery

Orchestration tools: **Docker Swarm**, **Kubernetes**.

Docker Swarm (Native Orchestrator)

What is Swarm?

- Docker's **native clustering & orchestration tool**.
- Groups multiple Docker hosts into a **swarm cluster**.
- Provides **service scaling, load balancing, and rolling updates**.

Key Concepts

- **Node** → Machine in swarm (Manager or Worker).
- **Service** → Defines tasks (containers).
- **Task** → A single running container.
- **Overlay Network** → Allows services on different nodes to communicate.

Initialize Swarm

```
docker swarm init
```

Get join token (for worker nodes):

```
docker swarm join-token worker
```

Creating Services

```
docker service create --name myweb -p 8080:80 nginx
```

Creates an nginx service running in swarm mode.

List services: `docker service ls`

Scale services: `docker service scale myweb=5`

Update service: `docker service update --image nginx:1.21 myweb`

Remove service: `docker service rm myweb`

Stack Deployments

Stacks = multi-service applications (like Compose but for Swarm).

```
docker stack deploy -c docker-compose.yml mystack
```

Check stack services:

```
docker stack services mystack
```

Swarm Summary: Good for learning, but in industry → **Kubernetes is more widely used**.

Kubernetes (K8s) – Quick Intro

Kubernetes is the **most popular orchestration platform**.

Core Concepts

- **Pod** → Smallest deployable unit (can contain 1+ containers).
- **Deployment** → Defines how many pods, scaling, rolling updates.
- **Service** → Exposes Pods to network (ClusterIP, NodePort, LoadBalancer).
- **Namespace** → Logical separation of resources.

Example Deployment

nginx-deployment.yml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
```

```
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        ports:
        - containerPort: 80
```

Deploy:

```
kubectl apply -f nginx-deployment.yml
kubectl get pods
```

Expose service:

```
kubectl expose deployment nginx-deployment --type=NodePort --port=80
```

Why Kubernetes?

- Cloud-native standard.
- Auto-scaling, rolling updates, self-healing.
- Works on all major cloud providers.

CI/CD with Docker

CI/CD (Continuous Integration / Deployment) pipelines often **build, test, and push Docker images** automatically.

Example: GitHub Actions

.github/workflows/docker.yml

```
name: Docker CI
```

```
on:
  push:
    branches: [ "main" ]
```

```
jobs:
```

```
build:
  runs-on: ubuntu-latest
  steps:
  - uses: actions/checkout@v2
```

```
  - name: Set up Docker Buildx
    uses: docker/setup-buildx-action@v2
```

```
  - name: Log in to Docker Hub
    uses: docker/login-action@v2
    with:
      username: ${{ secrets.DOCKER_USERNAME }}
      password: ${{ secrets.DOCKER_PASSWORD }}
```

```
  - name: Build and push Docker image
    uses: docker/build-push-action@v3
    with:
      push: true
      tags: mydockerhubuser/myapp:latest
```

On every push → builds Docker image → pushes to Docker Hub.

GitLab CI Example

.gitlab-ci.yml

```
stages:
 - build
```

```
build_image:
  stage: build
  script:
    - docker build -t registry.gitlab.com/myuser/myapp:latest .
    - docker push registry.gitlab.com/myuser/myapp:latest
```

Common CI/CD Steps with Docker

1. Checkout code.
2. Build Docker image.
3. Run tests inside container.
4. Push image to registry (Docker Hub, AWS ECR, GitLab, GCP, etc.).
5. Deploy to staging/production (Swarm, Kubernetes, or cloud).

Cloud Deployment with Docker

- **AWS ECS/Fargate** → Runs containers without managing servers.

- **Google Cloud Run** → Deploy containers directly (serverless).
- **Azure Container Instances** → Run container without VM.
- **DigitalOcean Apps / Render / Heroku** → Push Docker image → deploy in minutes.

Practice for Day 6

1. Initialize a Docker Swarm locally, deploy nginx service, and scale to 3 replicas.
2. Write a simple Compose file → deploy it as a **Swarm stack**.
3. Install Minikube (or use Play with Kubernetes online) → run a deployment with 3 pods.
4. Expose Kubernetes service → access via NodePort.
5. Create GitHub Actions workflow to **build & push Docker image** to Docker Hub.
6. Deploy your Dockerized app to cloud (AWS ECS, DigitalOcean, or Render).

Summary (Day 6)

- **Swarm**: Docker's native orchestrator (services, stacks, scaling).
- **Kubernetes**: Industry standard (pods, deployments, services).
- **CI/CD with Docker**: Automate build & push using GitHub Actions or GitLab CI.
- **Cloud Deployment**: Run containers on AWS/GCP/Azure/DigitalOcean.

#Day-07 (Real-world Projects & deployment)

Final Capstone Workflow

Here's the **end-to-end lifecycle** you should be comfortable with:

1. **Containerize your application** with a Dockerfile.
2. **Run locally** and test with docker run.
3. **Orchestrate multi-container setup** with Docker Compose.
4. **Push image to Docker Hub (or private registry)**.
5. **Automate builds with CI/CD** (GitHub Actions / GitLab).
6. **Deploy to cloud** (AWS ECS, DigitalOcean, Render, etc.).
7. **Add monitoring & logging**.

Example Full-Stack App Setup

Let's say you're deploying a **Node.js + React + PostgreSQL** app.

Dockerfile for Backend (Node.js)

```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install –production
```

```
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

Dockerfile for Frontend (React)

```
FROM node:18-alpine as build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
```

```
FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Docker Compose Setup

docker-compose.yml

```
version: "3.9"
services:
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      - backend
```

```
  backend:
    build: ./backend
    ports:
      - "5000:5000"
    environment:
      - DB_HOST=db
      - DB_USER=postgres
      - DB_PASSWORD=secret
      - DB_NAME=myapp
    depends_on:
      - db
```

```
  db:
    image: postgres:15
    restart: always
    environment:
```

```
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: secret
POSTGRES_DB: myapp
  volumes:
    - dbdata:/var/lib/postgresql/data
```

```
volumes:
 dbdata  :
```

Run everything with:

```
docker compose up –d
```

Push Images to Registry

Tag image:

```
docker tag myapp-backend mydockerhubuser/myapp-backend:1.0
docker push mydockerhubuser/myapp-backend:1.0
```

Do the same for frontend.

CI/CD Pipeline

Use GitHub Actions to **build & push images automatically**:

.github/workflows/docker.yml

```
name: Build and Push Docker Images
```

```
on:
 push:
  branches: [ "main" ]
```

```
jobs:
 build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v3
```

```
    - name: Log in to Docker Hub
      uses: docker/login-action@v2
      with:
        username: ${{ secrets.DOCKER_USERNAME }}
        password: ${{ secrets.DOCKER_PASSWORD }}
```

```
  - name: Build and push backend
    uses: docker/build-push-action@v3
    with:
      context: ./backend
      push: true
      tags: mydockerhubuser/myapp-backend:latest
```

```
  - name: Build and push frontend
    uses: docker/build-push-action@v3
    with:
      context: ./frontend
      push: true
      tags: mydockerhubuser/myapp-frontend:latest
```

Deployment Options

(a) Cloud VM (Simple Approach)

- Deploy on **AWS EC2, DigitalOcean Droplet, Linode, etc.**
- Install Docker & Compose → run `docker compose up -d`.

(b) Managed Container Services

- **AWS ECS / Fargate**: Serverless container hosting.
- **Google Cloud Run**: Deploy containers as serverless apps.
- **Azure Container Instances**: Quick single-container runs.

(c) Kubernetes (Advanced)

- Use your pushed images inside a **K8s deployment**.
- Add **Ingress** for frontend & backend routing.

Monitoring & Logging

- Use `docker logs -f` for quick debugging.
- For production:
  - **Prometheus + Grafana** → metrics visualization.
  - **ELK/EFK Stack** (Elasticsearch + Fluentd/Fluentbit + Kibana) → logs.
  - **cAdvisor** → container resource monitoring.

Project Ideas

1. **Blog App**

    - Frontend (React) + Backend (Node.js) + PostgreSQL
    - Deploy with Docker Compose
2. **E-commerce Clone**
    - Frontend (Vue/React) + Backend (Django/Node) + DB + Redis (cache)
    - Add CI/CD pipeline
3. **Real-time Chat App**
    - Backend (Node + Socket.IO) + Redis (pub/sub) + Frontend
    - Deploy on cloud
4. **Portfolio/Resume Site**
    - Simple React/Next.js app
    - Containerize & deploy via GitHub Actions → Docker Hub → Cloud

Final Checklist

Can you:

- Write a **Dockerfile** for any app?
- Build & run container locally?
- Create a **multi-service setup** with Compose?
- Use **volumes** for persistence & **networks** for service communication?
- Push images to Docker Hub?
- Automate build & push with CI/CD?
- Deploy on a cloud server?
- Add healthchecks, resource limits, restart policies?

If **yes** → you've mastered Docker fundamentals

Summary (Day 7)

- Combine everything into **end-to-end project workflow**.
- Containerize frontend, backend, database.
- Use Docker Compose for local orchestration.
- Automate builds with GitHub Actions / GitLab CI.
- Push images → Deploy on cloud.
- Add monitoring & logging for production readiness.

Congratulations — completing this means you've mastered Docker for real-world projects!

MADE BY ADNAN QURESHI